Computational Pipeline — Complete Code Reference

Microplastic Degradation Research Project | Researcher: Arjun Popuri | April 2026

> *This document contains the exact code used in every phase of the computational pipeline with explanations of what each block does. Code shown in dark background. Blue side-bar boxes explain the biological and computational purpose of each block.*

Phase 1 — Pepsin Cleavage Site Mapping (step1_pepsin_cleavage.py)

Block 1 — Imports and file setup

```
import os, urllib.request
import pandas as pd
from pyteomics import parser
from Bio import SeqIO

SCRIPT_DIR   = os.path.dirname(os.path.abspath(__file__))
PROJECT_ROOT = os.path.dirname(SCRIPT_DIR)
RESULTS_DIR  = os.path.join(PROJECT_ROOT, 'results', 'step1_pepsin')
os.makedirs(RESULTS_DIR, exist_ok=True)
```

> *Imports four libraries: os for file handling, urllib for downloading sequences, pandas for saving TSV results, pyteomics for applying pepsin cleavage rules, BioPython for reading FASTA files. Sets up folder paths so outputs save to the correct location.*

Block 2 — Download IsPETase sequence from UniProt

```
UNIPROT_ID = 'A0A0K8P6T7'
url = f'https://rest.uniprot.org/uniprotkb/{UNIPROT_ID}.fasta'
urllib.request.urlretrieve(url, FASTA_OUT)
record = SeqIO.read(FASTA_OUT, 'fasta')
wt_seq = str(record.seq)
print(f'Sequence length: {len(wt_seq)} aa')  # 290
```

> *Downloads the 290 amino acid wild-type IsPETase sequence from UniProt accession A0A0K8P6T7. BioPython SeqIO.read parses the FASTA and extracts the sequence as a Python string.*

Block 3 — Apply pepsin pH 1.3 cleavage rule

```
peptides = parser.cleave(
    wt_seq,
    rule=parser.expasy_rules['pepsin pH1.3'],
    missed_cleavages=0
)
```

```
 # Record each cleavage site
for pep in peptides:
    start = wt_seq.index(pep, pos)
    p1_residue = wt_seq[start - 1]  # The cut residue
    rows.append({'position': start, 'P1_residue': p1_residue, ...})
```

Applies the ExPASy pepsin pH 1.3 cleavage rule via pyteomics. Pepsin cleaves at F and L residues at the P1 position. The loop records position, P1 residue identity, and context window for all 35 predicted cleavage sites.

Phase 2 — Structure Analysis (step2_structure_analysis.py)

Block 1 — Download PDB and compute SASA

```
url = 'https://files.rcsb.org/download/5XJH.pdb'
urllib.request.urlretrieve(url, PDB_OUT)

parser  = PDBParser(QUIET=True)
structure = parser.get_structure('5XJH', PDB_OUT)
chain_A = structure[0]['A']

sr = ShrakeRupley()
sr.compute(structure, level='R')  # Compute at residue level
```

Downloads the 1.5 Angstrom IsPETase crystal structure. ShrakeRupley computes solvent accessible surface area for all 263 residues in chain A. Each residue gets a .sasa attribute with its absolute surface area.

Block 2 — Annotate residues with relative SASA and triad distance

```
for res in std_residues:
    asa     = res.sasa
    rel_asa = asa / MAX_ASA[resname]  # Tien 2013 max values
    ca      = res['CA'].get_vector().get_array()
    min_dist = float(min(np.linalg.norm(ca - c) for c in cat_ca))
    rows.append({'is_surface': rel_asa >= 0.25,
                 'near_active_site': min_dist < 8.0, ...})
```

For each residue: divides absolute SASA by Tien 2013 maximum to get relative SASA from 0-1. Computes minimum Euclidean distance to any catalytic triad alpha-carbon using numpy. Flags surface-exposed (>= 0.25) and near-active-site (< 8 A) residues.

Block 3 — Screen disulfide bond candidates

```
from itertools import combinations
for i, j in combinations(range(len(elig)), 2):
    cb_dist = float(np.linalg.norm(cb1 - cb2))
```

```
    if not (3.5 <= cb_dist <= 5.5):
        continue  # Outside geometric feasibility range
    pair_rows.append({'CB_CB_dist_A': round(cb_dist, 3), ...})
```

Tests every pair of surface-exposed non-Gly/Pro/Cys residues for geometric disulfide feasibility. Cbeta-Cbeta distance of 3.5-5.5 Angstroms is the standard criterion. Found 16 candidates; best was N233C+S282C at 3.869 Angstroms.

Phase 3 — Variant Design (step3_design_variant.py)

Block 1 — Define mutations and verify wild-type residues

```
MUTATIONS = [
    (51,  'T', 'C', 'disulfide', 'Forms S-S bond with T72C'),
    (59,  'R', 'E', 'charge-reversal', 'rel_ASA=0.68'),
    # ... all 8 mutations
    (285, 'R', 'E', 'charge-reversal', 'rel_ASA=0.40'),
]

for pos, wt_aa, new_aa, category, rationale in MUTATIONS:
    actual = seq_map[pos]
    assert actual == wt_aa, f'Mismatch at {pos}'
```

Defines all 8 mutations as tuples. Before applying any substitution, verifies that the actual amino acid at each position matches the expected wild-type residue. Raises an error and stops if any mismatch is found — protecting against numbering errors.

Block 2 — Apply mutations and verify catalytic triad

```
eng_list = list(wt_seq)
for pos, wt_aa, new_aa, category, rationale in MUTATIONS:
    eng_list[pos - 1] = new_aa  # 0-indexed
eng_seq = ''.join(eng_list)

# Verify triad untouched
for pos, expected_aa in CATALYTIC_TRIAD.items():
    assert eng_seq[pos - 1] == expected_aa
```

Converts sequence to a mutable list, applies each substitution at position-1 (0-indexed), joins back to string. Post-mutation assertions confirm Ser160, Asp206, His237 are unchanged and sequence length is still 290 aa.

Phase 4 — Codon Optimization (step4_codon_optimization.py)

Block 1 — Sharp and Li 1987 codon table and back-translation

```
ECOLI_OPTIMAL = {
    'A': 'GCG', 'R': 'CGT', 'N': 'AAC', 'D': 'GAT', 'C': 'TGC',
    'E': 'GAA', 'G': 'GGT', 'L': 'CTG', 'K': 'AAA', ...
    # All 20 amino acids mapped to fastest E. coli codon
}

def back_translate(aa_seq):
```

```
    return ''.join(ECOLI_OPTIMAL[aa] for aa in aa_seq)

translated = str(Seq(dna_seq).translate(table=11, to_stop=True))
assert translated == aa_seq  # Verify no errors
```

Maps each amino acid to its highest-CAI codon in E. coli K-12. back_translate builds the 870 bp DNA string. The verification re-translates the DNA and compares to the original protein — both passed with CAI = 1.000.

Phase 5 — Cloning Constructs (step5_cloning_constructs.py)

Block 1 — Assemble and verify 903 bp insert

```
NDEI_SITE  = 'CATATG'             # NdeI site + ATG start codon
HIS6_TAG   = 'CACCACCACCACCACCAC' # Encodes HHHHHH
STOP_CODON = 'TAA'
XHOI_SITE  = 'CTCGAG'

insert = NDEI_SITE + HIS6_TAG + cds + STOP_CODON + XHOI_SITE
assert len(insert) == 903
assert 'CATATG' not in cds  # No internal NdeI
assert 'CTCGAG' not in cds  # No internal XhoI

protein = str(Seq(translatable).translate(table=11, to_stop=True))
assert protein.startswith('MHHHHHH')  # Correct His-tag
```

Concatenates five elements to produce the 903 bp insert. Verifies exact length, no internal restriction sites (which would cause cloning to fail), and correct MHHHHHH N-terminal tag from in silico translation. All checks passed for both constructs.